Architecture 1001: x86-64 Assembly ArrayLocalVariable.c

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

ArrayLocalVariable.c

Adding and accessing an array local variable

```
short main(){
    short a;
    int b[6];
    long long c;
    a = 0xbabe;
    c = 0xba1b0ab1edb100d;
    b[1] = a;
    b[4] = b[1] + c;
    return b[4];
}
```

```
main:
0000000140001000  sub          rsp,38h
0000000140001004  mov          eax,0FFFFBABEh
0000000140001009  mov          word ptr [rsp],ax
000000014000100D  mov          rax,0BA1B0AB1EDB100Dh
0000000140001017  mov          qword ptr [rsp+8],rax
000000014000101C  mov          eax,4
0000000140001021  imul         rax,rax,1
0000000140001025  movsx        ecx,word ptr [rsp]
0000000140001029  mov          dword ptr [rsp+rax+10h],ecx
000000014000102D  mov          eax,4
0000000140001032  imul         rax,rax,1
0000000140001036  movsxd       rax,dword ptr [rsp+rax+10h]
000000014000103B  add          rax,qword ptr [rsp+8]
0000000140001040  mov          ecx,4
0000000140001045  imul         rcx,rcx,4
0000000140001049  mov          dword ptr [rsp+rcx+10h],eax
000000014000104D  mov          eax,4
0000000140001052  imul         rax,rax,4
0000000140001056  movzx        eax,word ptr [rsp+rax+10h]
000000014000105B  add          rsp,38h
000000014000105F  ret
```

ArrayLocalVariable.c

Adding and accessing an array local variable

```
short main(){
    short a;
    int b[6];
    long long c;
    a = 0xbabe;
    c = 0xba1b0ab1edb100d;
    b[1] = a;
    b[4] = b[1] + c;
    return b[4];
}
```

```
main:
0000000140001000  sub          rsp,38h
0000000140001004  mov          eax,0FFFFBABEh
0000000140001009  mov          word ptr [rsp],ax
000000014000100D  mov          rax,0BA1B0AB1EDB100Dh
0000000140001017  mov          qword ptr [rsp+8],rax
000000014000101C  mov          eax,4
0000000140001021  imul         rax,rax,1
0000000140001025  movsx        ecx,word ptr [rsp]
0000000140001029  mov          dword ptr [rsp+rax+10h],ecx
000000014000102D  mov          eax,4
0000000140001032  imul         rax,rax,1
0000000140001036  movsxd       rax,dword ptr [rsp+rax+10h]
000000014000103B  add          rax,qword ptr [rsp+8]
0000000140001040  mov          ecx,4
0000000140001045  imul         rcx,rcx,4
0000000140001049  mov          dword ptr [rsp+rcx+10h],eax
000000014000104D  mov          eax,4
0000000140001052  imul         rax,rax,4
0000000140001056  movzx        eax,word ptr [rsp+rax+10h]
000000014000105B  add          rsp,38h
000000014000105F  ret
```



ArrayLocalVariable.c

Adding and accessing an array local variable

```
short main(){
    short a;
    int b[6];
    long long c;
    a = 0xbabe;
    c = 0xba1b0ab1edb100d;
    b[1] = a;
    b[4] = b[1] + c;
    return b[4];
}
```

```
main:
0000000140001000  sub          rsp,38h
0000000140001004  mov          eax,0FFFFBABEh
0000000140001009  mov          word ptr [rsp],ax
000000014000100D  mov          rax,0BA1B0AB1EDB100Dh
0000000140001017  mov          qword ptr [rsp+8],rax
000000014000101C  mov          eax,4
0000000140001021  imul         rax,rax,1
0000000140001025  movsx        ecx,word ptr [rsp]
0000000140001029  mov          dword ptr [rsp+rax+10h],ecx
000000014000102D  mov          eax,4
0000000140001032  imul         rax,rax,1
0000000140001036  movsxd       rax,dword ptr [rsp+rax+10h]
000000014000103B  add          rax,qword ptr [rsp+8]
0000000140001040  mov          ecx,4
0000000140001045  imul         rcx,rcx,4
0000000140001049  mov          dword ptr [rsp+rcx+10h],eax
000000014000104D  mov          eax,4
0000000140001052  imul         rax,rax,4
0000000140001056  movzx        eax,word ptr [rsp+rax+10h]
000000014000105B  add          rsp,38h
000000014000105F  ret
```





IMUL - Signed Multiply

- FYI, Visual Studio seems to have a predilection for imul over mul (unsigned multiply). You'll see it showing up in places you expect mul
  - I haven't been able to get it to generate normal mul for simple examples
- Three forms. One, two, or three operands:
  - imul r/mX — rdx:rax = rax * r/mX
  - imul reg, r/mX — reg = reg * r/mX
  - imul reg, r/mX, immediate — reg = r/mX * immediate
- **Three** operands? Possibly the only "basic" instruction (meaning non-added-on-instruction-set(MMX,AVX,AEX,VMX,etc)) of it's kind?

IMUL - ☝🤓 "Well actually..."

- There are 13 forms, spread across 3 *groups*
  - Can also be seen as 5 groups depending on how you slice the 3-operand groups
  - So I'm going to show you 5 groups to not have too much on a single slide

IMUL Group 1 - Single Operand

IMUL r/m8	AX = AL * r/m8
IMUL r/m16	DX:AX = AX * r/m16
IMUL r/m32	EDX:EAX = EAX * r/m32
IMUL r/m64	RDX:RAX = RAX * r/m64

IMUL Group 2 - Two Operands

IMUL r16, r/m16	r16 = r16 * r/m16
IMUL r32, r/m32	r32 = r32 * r/m32
IMUL r64, r/m64	r64 = r64 * r/m64

⚠️TRUNCATION WARNING!⚠️

IMUL Group 3
Three Operands, 8 Bit Immediate

IMUL r16, r/m16, imm8	r16 = r/m16 * sign-extended imm8
IMUL r32, r/m32, imm8	r32 = r/m32 * sign-extended imm8
IMUL r64, r/m64, imm8	r64 = r/m64 * sign-extended imm8

⚠️TRUNCATION WARNING!⚠️

IMUL Group 4
Three Operands, 16 Bit Immediate

IMUL r16, r/m16, imm16	r16 = r/m16 * imm16

⚠️TRUNCATION WARNING!⚠️

IMUL Group 5
Three Operands, 32 Bit Immediate

IMUL r32, r/m32, imm32	r32 = r/m32 * imm32
IMUL r64, r/m64, imm32	r64 = r/m64 * sign-extended imm32

⚠️TRUNCATION WARNING!⚠️

IMUL Example 1

IMUL r/m8	AX = AL * r/m8

Registers Before

r12	**0x84**
rax	0x609966C1A977E1**77**

IMUL Example 1

IMUL r/m8	AX = AL * r/m8

Registers Before

r12	**0x84**
rax	0x609966C1A977E177

"imul r12b"

IMUL Example 1

IMUL r/m8	AX = AL * r/m8

Registers Before

r12	**0x84**
rax	0x609966C1A977E1**77**

"imul r12b"

r12	0x84
rax	0x609966C1A977**C65C**

Registers After

IMUL Example 2

IMUL r32, r/m32	r32 = r32 * r/m32

Registers Before

r12	**0x84**
rax	0x609966C1**A977E177**

"imul r12d, eax"

IMUL Example 2

IMUL r32, r/m32	r32 = r32 * r/m32

Registers Before

r12	**0x84**
rax	0x609966C1**A977E177**



"imul r12d, eax"



r12	0x00000000**61D0415C**
rax	0x609966C1A977E177

Registers After

IMUL Example 3

IMUL r64, r/m64, imm32	r64 = r/m64 * sign-extended imm32

Registers Before

r12	**0x84**
rax	**0x609966C1A977E177**

"imul r12, rax, 12341234"

IMUL Example 3

IMUL r64, r/m64, imm32	r64 = r/m64 * sign-extended imm32

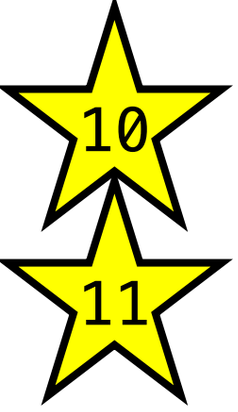
Registers Before

r12	**0x84**
rax	**0x609966C1A977E177**

"imul r12, rax, 12341234"

r12	**0xE5A3577504602A2C**
rax	0x609966C1A977E177

Registers After



MOVZX - Move with zero extend
MOVSX - Move with sign extend

- Used to move small values (from smaller types) into larger registers (holding larger types)
- Support same r->r, r->m, m->r, i->m, i->r forms as normal MOV
- “Zero extend” means the CPU unconditionally fills the high order bits of the larger register with zeros
- “Sign extend” means the CPU fills the high order bits of the destination larger register with whatever the sign bit is set to on the small value

MOVSXD - Move with sign extend

- MOVSX technically only sign extends from 8 or 16 bit values
- If you want to sign extend a 32 bit value to 64 bits, you need to use MOVSXD
  - There's no "MOVZXD", it's always just MOVZX
- I'm not going to give it its own star though 

MOVZX/MOVSX - examples

- `mov eax, 0xF00DFACE`
- `movzx rbx, eax`

  rbx = 0x00000000`F00DFACE
- `movsxd rbx, eax`

  rbx = 0xFFFFFFFF`F00DFACE

Because the sign bit (most significant bit) of 0xF00DFACE is 1



STOP
&&
STEP THROUGH
THE ASSEMBLY
&&
DRAW A STACK DIAGRAM!



STOP
&&
STEP THROUGH
THE ASSEMBLY
&&
DRAW A STACK DIAGRAM!

main()
frame

00000000`0014FE08	return address = 00000001`40001379
00000000`0014FE00	*16-byte-stack-alignment padding*
00000000`0014FDF8	undef
00000000`0014FDF0	undef`1edacacb
00000000`0014FDE8	undef
00000000`0014FDE0	ffffbabe`undef
00000000`0014FDD8	0ba1b0ab`1edb100d
00000000`0014FDD0	undef, 0xbabe (2 bytes)

main()
frame

00000000`0014FE08	return address = 00000001`40001379
00000000`0014FE00	*16-byte-stack-alignment padding*
00000000`0014FDFC	undef (alignment padding)
00000000`0014FDF8	undef (alignment padding)
00000000`0014FDF4	b[5] = undef
00000000`0014FDF0	b[4] = 1edacacb
00000000`0014FDEC	b[3] = undef
00000000`0014FDE8	b[2] = undef
00000000`0014FDE4	b[1] = ffffbabe
00000000`0014FDE0	b[0] = undef
00000000`0014FDDC	c (MSBs) = 0ba1b0ab
00000000`0014FDD8	c (LSBs) = 1edb100d
00000000`0014FDD4	undef (alignment padding)
00000000`0014FDD0	a = undef babe (2 bytes)

4-byte-granularity addresses

ArrayLocalVariable.c takeaways

- Local variables *need not be* stored on the stack in the same order they are defined in the high level language
- (In VS unoptimized code) Array access is typically done by multiplying the size of the array element (4 bytes for an int in this case), times the index that is desired to be access (indices 1 and 4 in this case)
- Moving a small value to a large register will result in a zero extend. Addition using signed values could result in a sign extend, if the arithmetic is done in a larger register

```
short main(){
    short a;
    int b[6];
    long long c;
    a = 0xbabe;
    c = 0xba1b0ab1edb100d;
    b[1] = a;
    b[4] = b[1] + c;
    return b[4];
}
```

```
sub     rsp,38h
mov     eax,0FFFFBABEh
mov     word ptr [rsp],ax
mov     rax,0BA1B0AB1EDB100Dh
mov     qword ptr [rsp+8],rax
mov     eax,4
imul    rax,rax,1
movsx   ecx,word ptr [rsp]
mov     dword ptr [rsp+rax+10h],ecx
mov     eax,4
imul    rax,rax,1
movsxd  rax,dword ptr [rsp+rax+10h]
add     rax,qword ptr [rsp+8]
mov     ecx,4
imul    rcx,rcx,4
mov     dword ptr [rsp+rcx+10h],eax
mov     eax,4
imul    rax,rax,4
movzx   eax,word ptr [rsp+rax+10h]
add     rsp,38h
ret
```

Instructions we now know (11)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- **IMUL**
- **MOVZX/MOVSX**

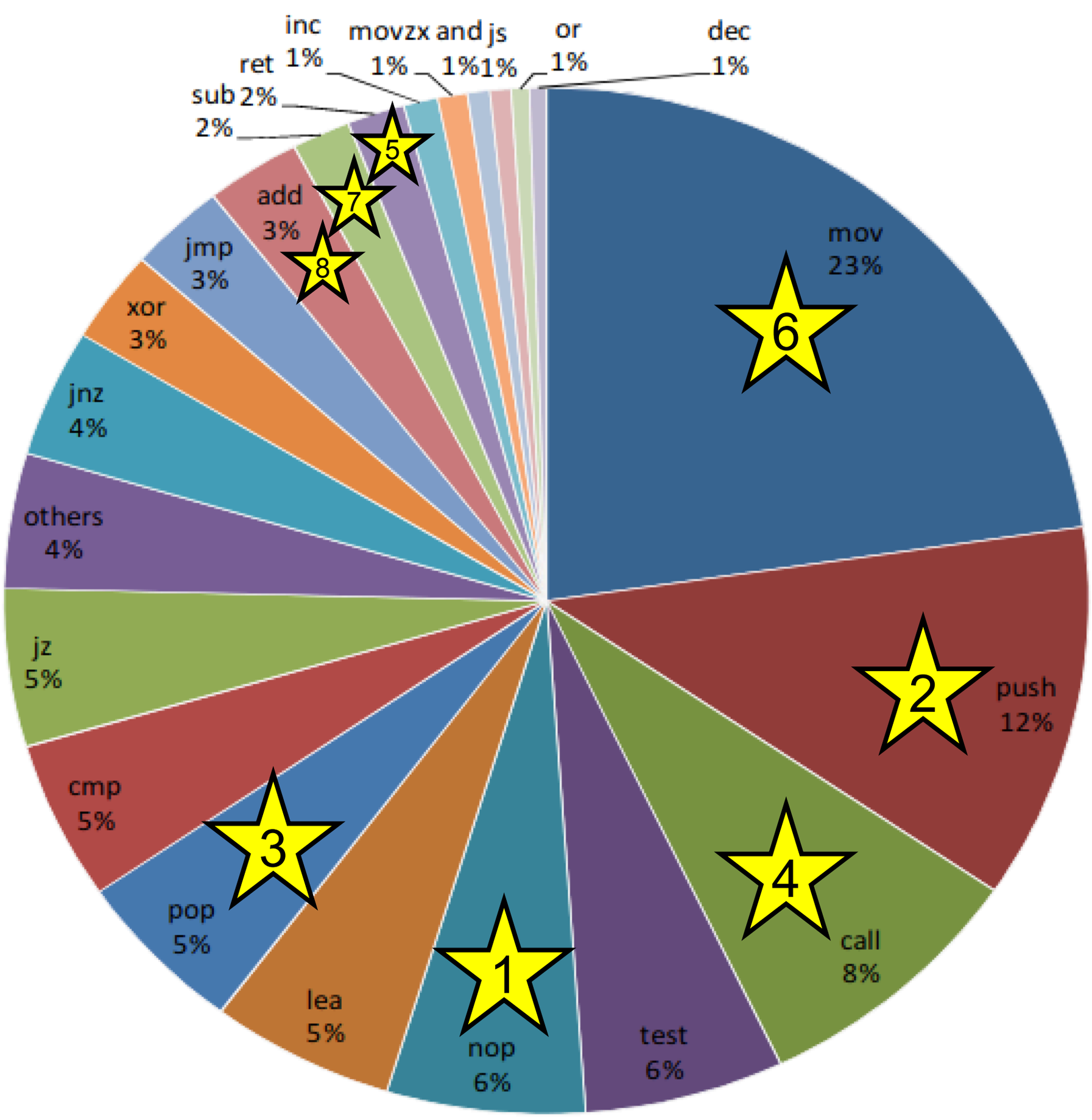


Fig. 2 Web browsers' instruction frequencies

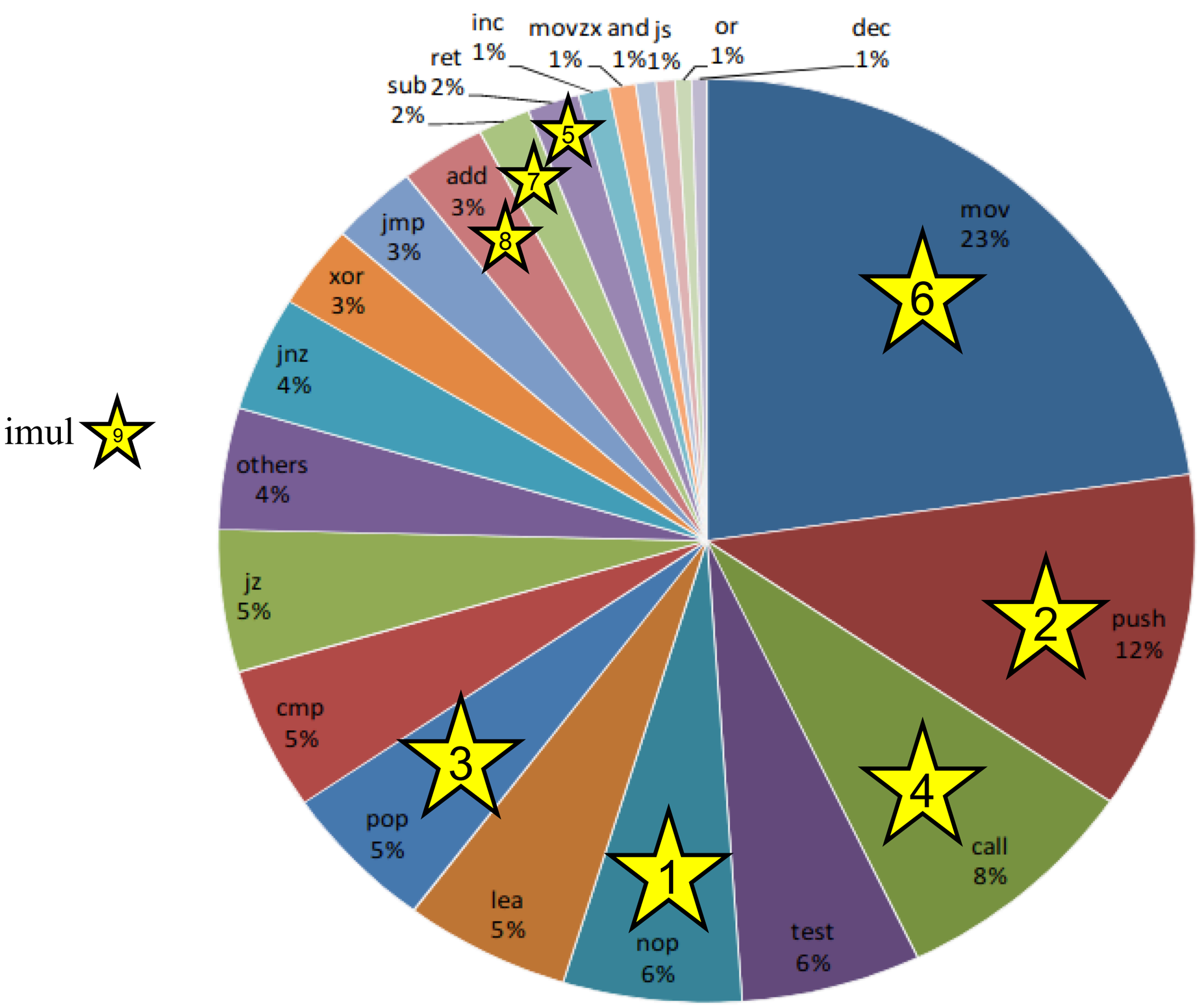


Fig. 2 Web browsers' instruction frequencies

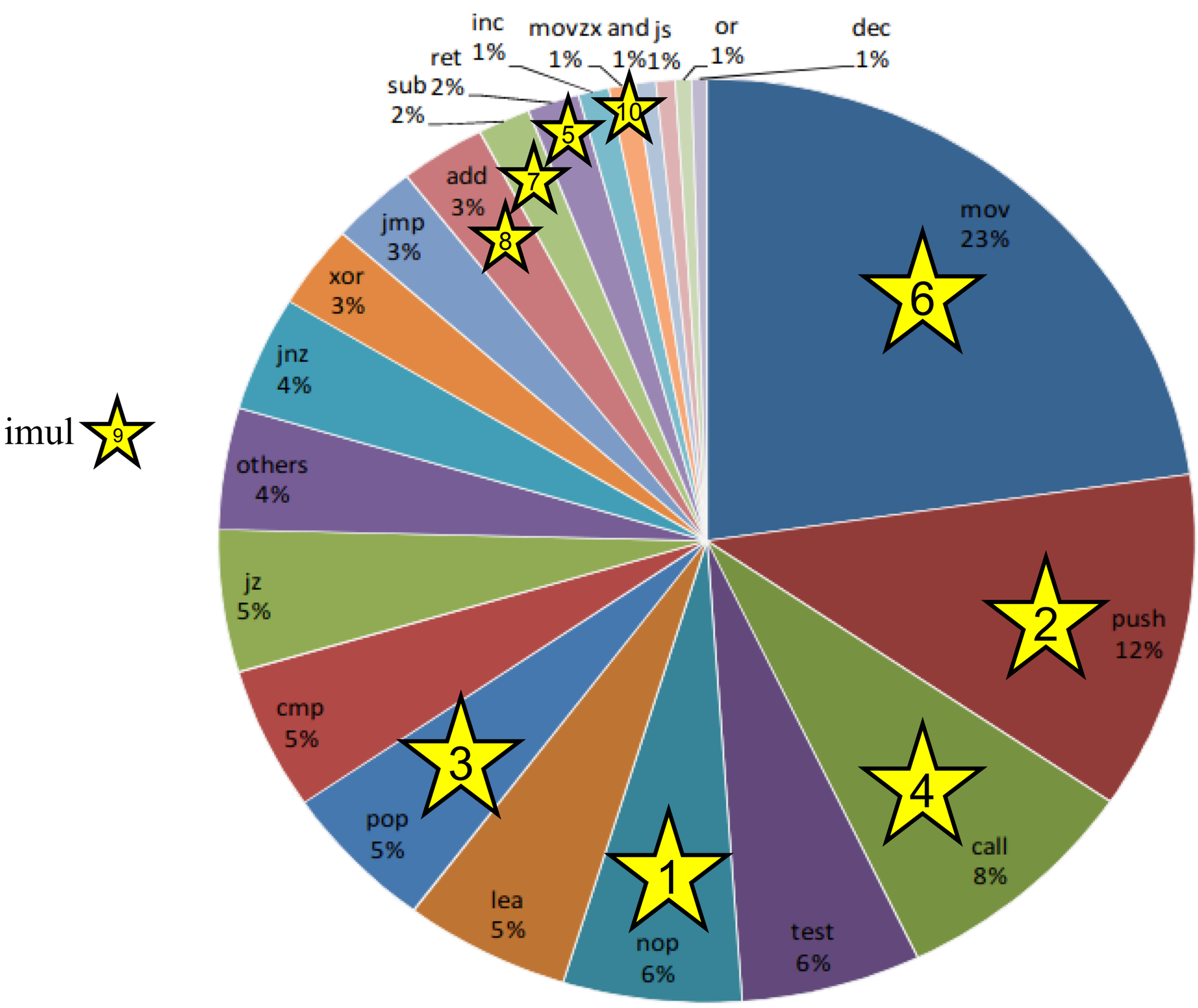


Fig. 2 Web browsers' instruction frequencies

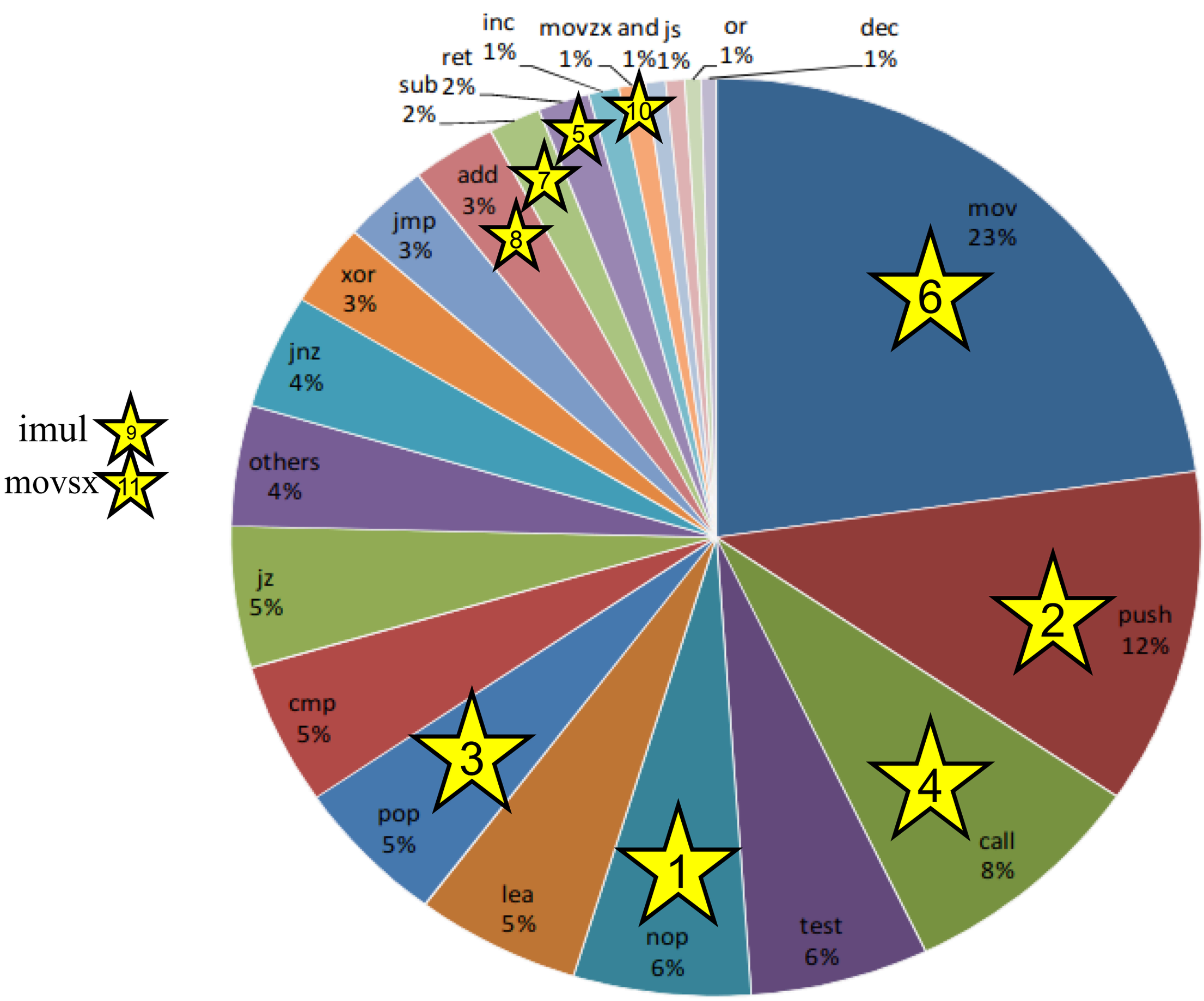


Fig. 2 Web browsers' instruction frequencies